

Project 5: A comprehensive comparison of image processing

Contents

1. 实验总览	1.
1.1. Experiment 1: BMP 图像读写对比	1.
1.1.1. Rust 图像读取/写入	1.
1.1.2. Python 图像读取/写入	2.
1.2. 实验设计	3.
1.3. 实验方法	3.
1.4. 实验结果	5.

§1. 实验总览

在 project4 中，我们已经搭建了一个比较完善的小型图像处理库，利用大量 C++ 的新特性对模块进行改进与封装。而编写出单按预期运行的代码并不能算是完成了所有的工作，我们希望这个处理库有着不输于现有实现框架的表现。因此，在这个实验中，我们将跨语言地对比使用 C++ 实现与 Rust 乃至 Python 等的流行的图像处理库的性能。

§1.1. Experiment 1: BMP 图像读写对比

无论要进行哪种处理方式，对图像的文件读写都是必不可少的步骤。目前机器学习中流行的图像数据集如 ImageNet[cite here], CIFAR-10[cite here] 都有着数万级别的训练图像和实例。如果是这种数量级的输入，读写时带来的性能损耗就无法忽略了。为了考察各语言图像编解码过程的时空占用情况，我们将开展不同大小、不同通道数的输入图像的读写基准测试。并尝试分析该过程的跨语言性能差异。

虽然我们在 project4 完全支持 PNG, JPEG 图像的读写，然而各标准库使用的 DCT 变换算法可能存在差异而引入测试噪声。因此我们仅对最直接的 BMP 文件开展测试。

§1.1.1. Rust 图像读取/写入

在 Rust 的测试中，我们考虑使用最流行的图像处理库 `image-rs` 作为基线来比较。我们会用到两个模块：`ImageReader` 和 `DynamicImage`。使用 `ImageReader` 读取图像文件的用例如下：

```
use image::{ImageReader};

fn main() {
```

```

let fname: String = /* Path to the image file */;
let img = ImageReader::open(fname)
    .expect("Failed to open image")
    .decode()
    .expect("Failed to decode image");
}

```

而 `DynamicImage` 用来产生指定大小的供写入的 BMP 图像，由于像素数据值对读写的影响较小，我们可以全部使用白色(255)来填充，以下是创建并保存一个 4096x4096 的白色灰度阶图像的代码：

```

use image::{DynamicImage, Rgba};

fn main() {
    let img = DynamicImage::new_luma8(4096, 4096);
    for x in 0..4096 {
        for y in 0..4096 {
            img.put_pixel(x, y, Rgba([255, 255, 255, 255]));
        }
    }
    img.save("output.bmp")
        .expect("Failed to save image");
}

```

§1.1.2. Python 图像读取/写入

在 Python 中，OpenCV 提供了更加简洁的接口来读取和写入图像。根据官方文档，`cv2.imread()` 函数可以读取图像文件并返回一个 NumPy 数组，而 `cv2.imwrite()` 函数可以将 NumPy 数组保存为图像文件。它们在文档里的描述如下：

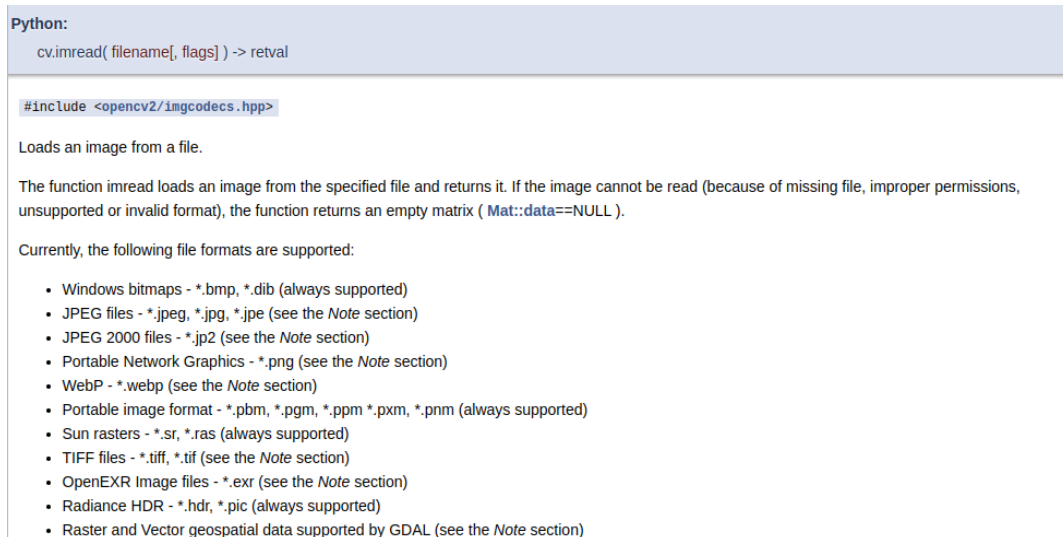


Figure 1: `cv2.imread()`

Python:

```
cv.imwrite( filename, img[, params] ) -> retval
```

```
#include <opencv2/imgcodecs.hpp>
```

Saves an image to a specified file.

The function `imwrite` saves the image to the specified file. The image format is chosen based on the filename extension (see `cv::imread` for the list of extensions). In general, only 8-bit unsigned (CV_8U) single-channel or 3-channel (with 'BGR' channel order) images can be saved using this function, with these exceptions:

- With OpenEXR encoder, only 32-bit float (CV_32F) images can be saved.
 - 8-bit unsigned (CV_8U) images are not supported.
- With Radiance HDR encoder, non 64-bit float (CV_64F) images can be saved.
 - All images will be converted to 32-bit float (CV_32F).
- With JPEG 2000 encoder, 8-bit unsigned (CV_8U) and 16-bit unsigned (CV_16U) images can be saved.
- With PAM encoder, 8-bit unsigned (CV_8U) and 16-bit unsigned (CV_16U) images can be saved.
- With PNG encoder, 8-bit unsigned (CV_8U) and 16-bit unsigned (CV_16U) images can be saved.
 - PNG images with an alpha channel can be saved using this function. To do this, create 8-bit (or 16-bit) 4-channel image BGRA, where the alpha channel goes last. Fully transparent pixels should have alpha set to 0, fully opaque pixels should have alpha set to 255/65535 (see the code sample below).
- With PGM/PPM encoder, 8-bit unsigned (CV_8U) and 16-bit unsigned (CV_16U) images can be saved.
- With TIFF encoder, 8-bit unsigned (CV_8U), 16-bit unsigned (CV_16U), 32-bit float (CV_32F) and 64-bit float (CV_64F) images can be saved.
 - Multiple images (vector of `Mat`) can be saved in TIFF format (see the code sample below).
 - 32-bit float 3-channel (CV_32FC3) TIFF images will be saved using the LogLuv high dynamic range encoding (4 bytes per pixel)

If the image format is not supported, the image will be converted to 8-bit unsigned (CV_8U) and saved that way.

If the format, depth or channel order is different, use `Mat::convertTo` and `cv::cvtColor` to convert it before saving. Or, use the universal `FileStorage` I/O functions to save the image to XML or YAML format.

Figure 2: `cv2.imwrite()`

由于我们的图像处理库的读写函数就是参照 OpenCV 设计的，因此上手起来也非常容易。

§1.2. 实验设计

为了尽可能全面地衡量各个语言在这个过程中的性能，我们将对图像形状参数从如下三个角度进行变化：

1. 图像的宽高: 读写的图像大小会从 128x128 到 8192x8192，每次倍增其宽度和高度。

进行这种变化的原因比较好理解，我们从 project3 剖析了 BMP 图像的存储格式和解析方法可以知道，解析像素值时会在列与行上进行遍历。因此预期读写的时间复杂度是 $O(N^2)$ ，而常数因子会与各种语言的实现方式有关，这也是我们对比的重点。

2. 图像的通道数: 鉴于我们在 project4 中实现了对 RGB, RGBA, Luma 等不同位深度图像的支持。

我们将对比 1, 3, 4 三种不同通道数的图像读写性能。

3. 图像的宽高比: 读写的图像大小固定为 1080x1080，我们将对比 240:4860, 360:3240, 720:1620, 810:1440, 1080:1080 一系列宽高比的图像读写。

§1.3. 实验方法

整个实验会在我的电脑上进行基准测试，硬件配置与环境如下：

Item	Details
System	6.8.0-52-generic Ubuntu 22.04.1 LTS
OS Kernel	5.15.167.4-microsoft-standard-WSL2
gcc-version	(Ubuntu 9.4.0-1ubuntu1 20.04.2) 9.4.0

g++-version	(Ubuntu 9.4.0-1ubuntu1 20.04.2) 9.4.0
cargo-version	1.86.0 (adf9b6ad1 2025-02-28)
rustc-version	1.86.0 (05f9846f8 2025-03-31)
conda-version	24.11.3
python-env-version	3.13.2
CPU Intel(R) Core(TM)	i5-13600KF CPU 5.10GHz
CPU-cores	14 cores
L1-cache-size	72 KB
L2-cache-size	2048 KB
L3-cache-size	24576 KB
Memory	31.968 GB

基准测试库我们分别选择的是

- C++: Google Benchmark[cite here]
- Rust: Criterion[cite here]
- Python: pytest-benchmark[cite here]

其中 Google Benchmark 和 Criterion 我们在前面的 project 有所提及，这次的使用也将是相似的流程。而 pytest-benchmark 和两个基准测试库比较类似，它使用独有的装饰器接口 `@pytest.mark.parametrize` 来参数化测试函数，并在函数中使用 `benchmark` 对象来进行多次基准测试，在报告中也会输出详细的统计数据。

§1.4. 实验结果

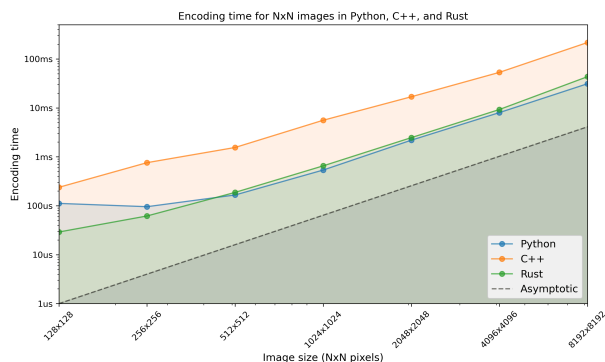


Figure 3: 单通道写入

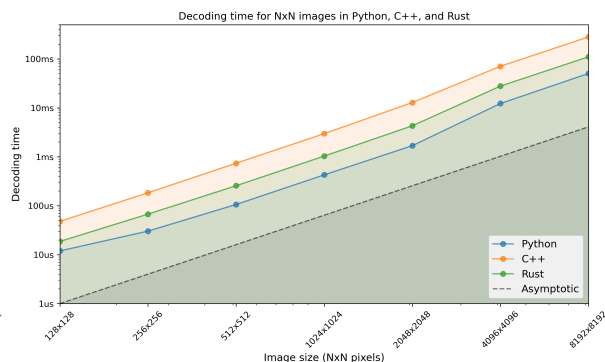


Figure 4: 单通道读取

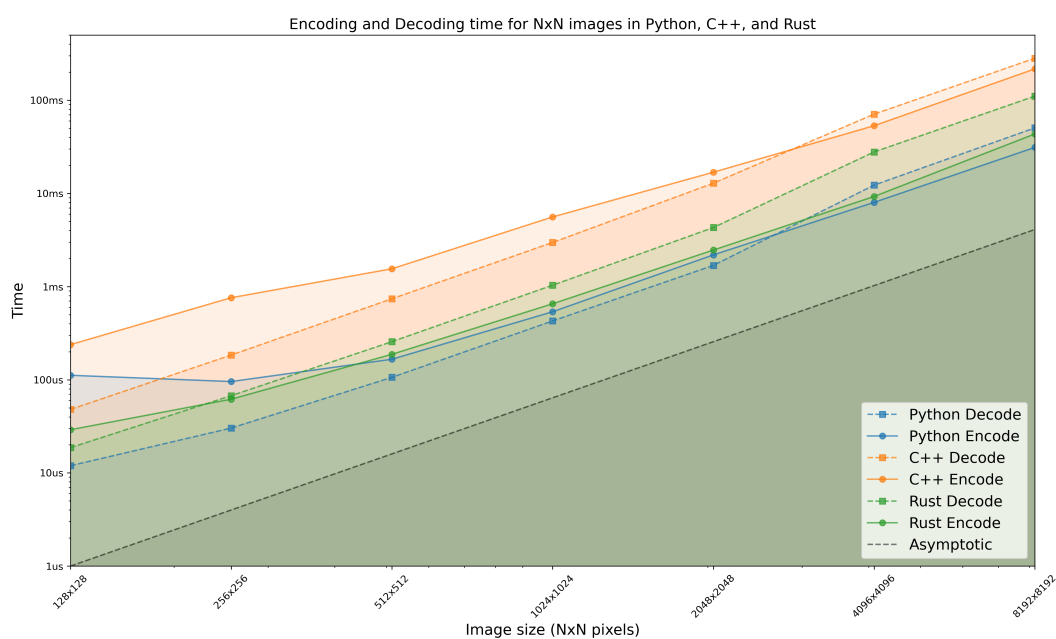


Figure 5: 单通道读写综合对比, 实线为写入, 虚线为读取

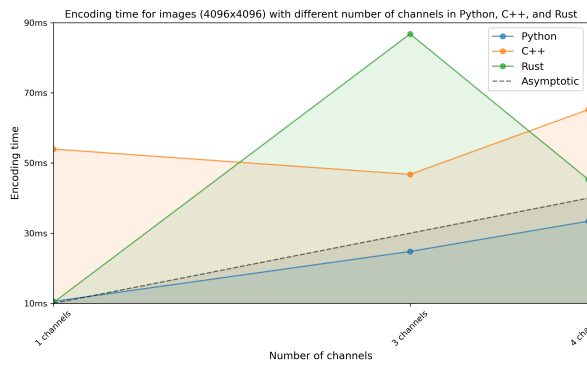


Figure 6: 多通道写入

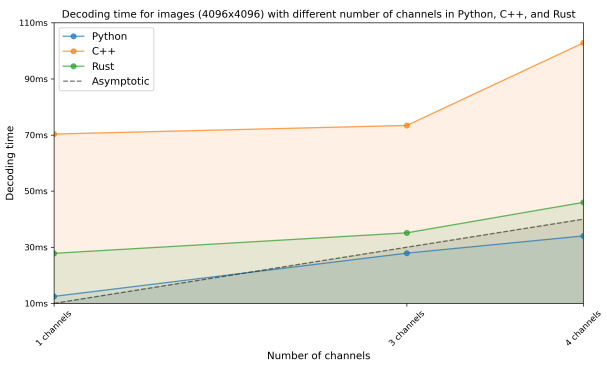


Figure 7: 多通道读取

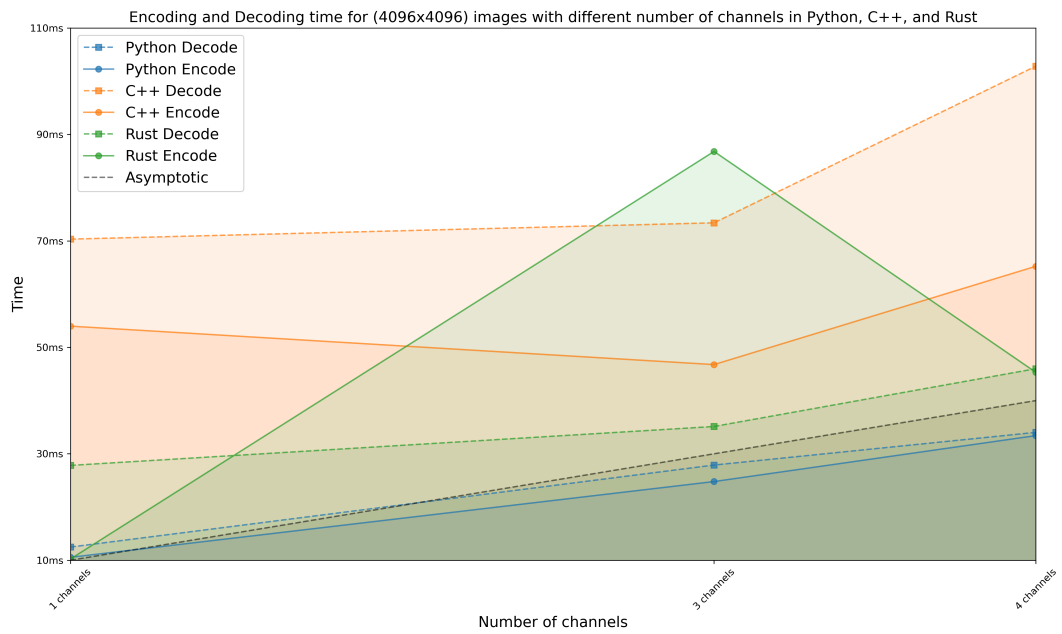


Figure 8: 多通道读写综合对比, 实线为写入, 虚线为读取